Metaheurísticas

Aula 5 — Algoritmos Genéticos e Evolução Diferencial

Raoni F. S. Teixeira · 1s/2026

Objetivos desta aula

Ao final desta aula, o aluno deve ser capaz de:

1. Explicar a diferença fundamental entre metaheurísticas de trajetória e algoritmos populacionais, identificando quando essa diferença é decisiva para o desempenho;
2. Implementar um Algoritmo Genético completo com operadores de seleção por torneio, cruzamento BLX-α e mutação polinomial;
3. Diagnosticar e tratar convergência prematura com elitismo, niching e reinjeção aleatória;
4. Implementar Evolução Diferencial nas estratégias DE/rand/1/bin e DE/best/1/bin e justificar a escolha entre elas com base na estrutura do problema;
5. Calibrar os parâmetros F e C_R da DE a partir de critérios analíticos e experimentais;
6. Comparar AG e DE em problemas combinatórios e contínuos com metodologia estatística rigorosa.

1 De trajetórias a populações

Os algoritmos do Módulo 2 — Hill Climbing, SA e Busca Tabu — operam sobre uma única solução que percorre o espaço de busca. Essa abordagem é eficiente em memória e simples de implementar, mas carrega uma limitação estrutural: informação sobre o espaço de busca é acumulada **implicitamente** — via temperatura ou lista tabu — e não pode ser diretamente reaproveitada para orientar buscas em regiões distintas.

Algoritmos populacionais superam essa limitação mantendo, a cada iteração, um conjunto de N_p soluções que interagem entre si. As soluções trocam informação, combinam atributos e geram descendentes que herdam características de múltiplos pais. Isso permite explorar paralelamente diversas regiões do espaço e emergir comportamentos coletivos que nenhum agente individual poderia produzir sozinho.

A mudança de paradigma tem um custo real: N_p avaliações de f por geração em vez de uma. Em problemas onde cada avaliação é cara — simulações físicas, fluxos de rede, modelos de elementos finitos — esse fator multiplicativo deve ser gerenciado na calibração de N_p e do critério de parada. A unidade de comparação correta continua sendo o **orçamento de avaliações** (NFE) total, não o número de gerações.

2 Algoritmos Genéticos

Algoritmos Genéticos (AG) foram introduzidos por John Holland (1975) como modelo computacional da evolução darwiniana. A analogia biológica é direta: soluções são **indivíduos**, o conjunto de soluções

é a **população**, a função objetivo é a **aptidão** (*fitness*), e cada iteração é uma **geração**. Operadores de **seleção**, **cruzamento** e **mutação** produzem gerações progressivamente mais aptas.

2.1 Representação

A escolha da representação é a decisão de projeto mais impactante em um AG. Ela determina quais operadores são naturais, quais restrições são fáceis de satisfazer e qual é a topologia efetiva do espaço de busca.

Representação binária. Cada solução é um vetor de bits $\boldsymbol{x} \in \{0,1\}^L$. Natural para problemas de decisão binária — o Problema da Mochila é o exemplo canônico: $x_i = 1$ se o item i está na mochila, $x_i = 0$ caso contrário. Problemas contínuos exigem conversão numérica com precisão controlada pelo número de bits por variável.

Representação real. Cada solução é um vetor $\boldsymbol{x} \in \mathbb{R}^n$. Natural para otimização contínua: parâmetros físicos, coordenadas, proporções. Elimina a artificialidade da codificação binária e viabiliza operadores mais eficientes como BLX-α e SBX.

Representação inteira. Para variáveis discretas não binárias: quantidades, índices, configurações de múltiplos níveis.

Representação permutacional. Para problemas de sequenciamento como o TSP: cada solução é uma permutação das cidades. Requer operadores especializados que preservem a validade da permutação — cruzamento OX (*Order Crossover*) e mutação por inversão de segmento.

> ⚠️ **Atenção**
> A representação deve ser escolhida **antes** dos operadores. Operadores projetados para representação binária são subótimos em representação real, e vice-versa. Um AG com representação inadequada raramente produz resultados competitivos, independentemente do ajuste fino dos demais parâmetros.

2.2 Seleção

O operador de **seleção** decide quais indivíduos geram descendentes. A **pressão seletiva** — quão fortemente os melhores indivíduos são favorecidos — controla o trade-off exploração/explotação no AG.

Seleção por roleta (*fitness proportionate*). A probabilidade de selecionar o indivíduo i é proporcional à sua aptidão:

$$p_i = \frac{f_i}{\sum_j f_j}$$

Funciona bem quando aptidões são positivas e bem distribuídas. Sofre de **dominância prematura**: se um indivíduo muito apto surge cedo, ele domina a roleta e a diversidade colapsa rapidamente.

Seleção por torneio. Amostram-se k indivíduos aleatoriamente; o melhor é selecionado. Repetido para cada posição de pai. É o método mais robusto na prática: a pressão seletiva é controlada diretamente por k (tipicamente $k = 2$ ou $k = 3$), sem depender da escala absoluta de f.

Seleção por ranking. Ordena a população por aptidão e atribui probabilidades de seleção baseadas no posto (rank), não no valor absoluto. Elimina dominância prematura e é insensível a escalonamentos de f.

2.3 Cruzamento

O **cruzamento** combina o material genético de dois pais para gerar novos indivíduos. É o operador central dos algoritmos genéticos (AG), responsável pela **recombinação de blocos construtivos** (*building blocks*): partes úteis de soluções distintas que, combinadas, podem gerar soluções superiores.

2.3.1 Cruzamento de um ponto (binário/inteiro)

Um ponto de corte k é sorteado em $\{1, ..., L-1\}$.

Pais
$P_1 = [a_1, a_2 m \operatorname{id} a_3, a_4, a_5]$
$P_2 = [b_1, b_2 m \operatorname{id} b_3, b_4, b_5]$

Filhos
$F_1 = [a_1, a_2, b_3, b_4, b_5]$
$F_2 = [b_1, b_2, a_3, a_4, a_5]$

2.3.2 Cruzamento uniforme

Cada gene é herdado independentemente:
- do pai 1 com probabilidade 0.5
- do pai 2 com probabilidade 0.5

É um operador mais **disruptivo**, indicado quando não há estrutura local clara nos genes.

2.3.2.1 Exemplo

Considere:

- $P_1 = [1, 0, 1, 1, 0]$
- $P_2 = [0, 1, 0, 0, 1]$

Máscara aleatória:

- $M = [1, 0, 1, 0, 1]$

Construção:

- $F = [1, 1, 1, 0, 0]$

Cada posição é independente, aumentando a variabilidade genética.

2.3.3 Cruzamento BLX-α (real)

Para cada dimensão d, o valor do filho é amostrado uniformemente em:

$$\left[x_d^{\{\min\}} - \alpha I_d, \quad x_d^{\{\max\}} + \alpha I_d\right]$$

onde:
- $x_d^{\{\min\}} = \min\left(p_{\{1d\}}, p_{\{2d\}}\right)$
- $x_d^{\{\max\}} = \max\left(p_{\{1d\}}, p_{\{2d\}}\right)$
- $I_d = x_d^{\{\max\}} - x_d^{\{\min\}}$

O parâmetro α controla a **exploração**. Um valor típico é $\alpha = 0.5$.

2.3.3.1 Exemplo (BLX-α)

Considere:

- $P_1 = [10, 20]$
- $P_2 = [12, 15]$
- $\alpha = 0.5$

Dimensão x:
- $I_x = 2$
- intervalo: $[9, 13]$

Dimensão y:
- $I_y = 5$
- intervalo: $[12.5, 22.5]$

Um possível filho:

- $F = [9.4, 21.2]$

Os valores podem extrapolar os pais, permitindo explorar novas regiões do espaço de busca.

2.3.4 Cruzamento SBX (Simulated Binary Crossover)

Imita o comportamento do cruzamento binário em variáveis reais.

O parâmetro η_c controla a distribuição:
- η_c alto $\longrightarrow$ filhos próximos (**exploração local**)
- η_c baixo $\longrightarrow$ filhos mais distantes (**exploração global**)

2.3.4.1 Exemplo (SBX)

Considere:

- $p_1 = 10$
- $p_2 = 12$
- $\eta_c = 2$
- $u = 0.3$

Cálculo de $b\eta_q$:

$$\beta_q = \begin{cases} (2u)^{\frac{1}{\eta_c+1}} & \text{se } u \leq 0.5 \\ \left(\frac{1}{2(1-u)}\right)^{\frac{1}{\eta_c+1}} & \text{se } u > 0.5 \end{cases}$$

Como $u = 0.3$:

$\left[b\eta_q \sim 0.843\right]$

Filhos:

$\left[F_1 = 0.5\left[(1 + b\eta_q)p_1 + (1 - b\eta_q)p_2\right]\right]$

$\left[F_2 = 0.5\left[(1 - b\eta_q)p_1 + (1 + b\eta_q)p_2\right]\right]$

Resultados:

- $F_1 \sim 10.16$

- $F_2 \sim 11.84$

2.3.5 Cruzamento OX (Order Crossover)

Usado em representações permutacionais (ex.: TSP).

Procedimento:
1. Seleciona-se um segmento do pai 1
2. Copia-se diretamente para o filho
3. Preenche-se o restante com a ordem do pai 2

2.3.5.1 Exemplo (OX)

Considere:

- $P_1 = [1, 2, 3, 4, 5, 6, 7, 8]$
- $P_2 = [5, 7, 2, 1, 8, 4, 6, 3]$

Segmento: posições 3 a 5

Passo 1:
- $F = [_, _, 3, 4, 5, _, _, _]$

Passo 2:

Elementos restantes de P_2 (removendo $\{3, 4, 5\}$):
- $[7, 2, 1, 8, 6]$

Preenchimento:

- $F = [7, 2, 3, 4, 5, 1, 8, 6]$

O operador preserva a validade da permutação e a ordem relativa dos elementos.

2.4 Mutação

A **mutação** introduz perturbações aleatórias em indivíduos, garantindo que o AG possa alcançar qualquer ponto do espaço e prevenindo convergência prematura. Deve ser aplicada com probabilidade baixa ($p_m \in [0.001, 0.05]$ por gene) para não destruir boas soluções sistematicamente.

Mutação bit-flip (binária). Cada bit é invertido com probabilidade p_m. Para o Knapsack: pode tornar a solução inviável (exceder W), requerendo reparo ou penalização.

Mutação gaussiana (real). Adiciona ruído: $x'_d = x_d + \mathcal{N}(0, \sigma_d)$. O desvio σ_d pode ser fixo ou adaptativo.

Mutação polinomial. Operador padrão em AGs de variáveis reais (Deb & Agrawal, 1995). Distribui a perturbação com probabilidade concentrada próximo ao valor original, com caudas que permitem saltos maiores. Controlada pelo parâmetro η_m: valores altos concentram a mutação perto do pai (explotação).

Mutação por inversão de segmento (permutacional). Inverte a ordem de um subsegmento aleatório da permutação — equivalente a um único movimento 2-opt. Mantém a validade da permutação por construção.

2.5 Elitismo e controle de diversidade

Elitismo. Garante que os e melhores indivíduos da geração t passem intactos para a geração $t+1$. Com $e = 1$, a sequência do melhor valor encontrado é monotonicamente não-crescente. É a extensão mais simples e mais impactante do AG básico.

Convergência prematura. Ocorre quando a população perde diversidade antes de encontrar o ótimo global: todos os indivíduos tornam-se semelhantes, cruzamentos deixam de gerar novidade e o AG estagna em ótimo local. Sintomas: variância da população próxima de zero, curva de convergência em platô precoce.

Contramedidas:

- **Reinjeção aleatória**: substituir os r piores indivíduos por soluções aleatórias a cada G gerações.
- **Fitness sharing**: reduzir artificialmente a aptidão de indivíduos semelhantes, penalizando a concentração em uma única região.
- **Crowding**: ao inserir um filho, substituir o indivíduo da população mais similar a ele, preservando diversidade geográfica.

2.6 Algoritmo completo

Algoritmo — Algoritmo Genético com Elitismo

```
AG(f, N_pop, N_gen, p_c, p_m, e):
  P ← {x_1, ..., x_{N_pop}}     // população inicial aleatória
  avaliar f(x_i) para todo i
  s* ← argmin f(P)

  para g = 1 até N_gen:
    elite ← melhores e indivíduos de P

    P_nova ← elite
    enquanto |P_nova| < N_pop:
      pai1, pai2 ← selecao_torneio(P)
      se uniforme(0,1) < p_c:
        filho1, filho2 ← cruzamento(pai1, pai2)
      senão:
        filho1, filho2 ← pai1, pai2
      filho1 ← mutacao(filho1, p_m)
      filho2 ← mutacao(filho2, p_m)
      P_nova ← P_nova ∪ {filho1, filho2}

    P ← P_nova
    avaliar f(x_i) para x_i ∈ P \ elite
    se f(argmin P) < f(s*): s* ← argmin P

  retorna s*
```

2.7 Parâmetros e guia de calibração

Parâmetro	Faixa típica	Impacto e guia
N_{pop}	20–200	Maior: mais diversidade, mais custo. Regra: $N_{\text{pop}} = 10n$ para n variáveis; reduzir se avaliação de f for cara
N_{gen}	100–2000	Definir pelo orçamento total: $N_{\text{pop}} \times N_{\text{gen}} = \text{NFE}$
p_c	0,6–0,9	$p_c = 0.8$ é ponto de partida seguro; reduzir em paisagens rugosas
p_m	0,001–0,05	Por gene. Regra: $p_m = \frac{1}{n}$; muito alta destrói building blocks
e (elitismo)	1–5	Valores > 5 reduzem diversidade; $e = 1$ é o mínimo recomendado
Torneio k	2–5	$k = 2$: baixa pressão (exploração); $k = 5$: alta pressão (explotação)

Table 1: Parâmetros do Algoritmo Genético e guia de calibração.

3 Evolução Diferencial

A **Evolução Diferencial** (Differential Evolution, DE) foi proposta por Storn e Price em 1995 e rapidamente se tornou um dos algoritmos mais competitivos para otimização contínua. Compartilha a estrutura populacional com o AG, mas o mecanismo de geração de candidatos é radicalmente diferente: em vez de cruzar pais selecionados por aptidão, a DE cria candidatos por **perturbação vetorial diferencial** — a diferença entre dois indivíduos aleatórios da população, escalada por um fator F e somada a um terceiro.

Esse mecanismo tem duas propriedades valiosas: (1) a magnitude da perturbação se adapta automaticamente à diversidade da população — quando a população converge, as diferenças encolhem e as perturbações ficam menores; (2) não requer nenhum parâmetro relacionado à escala de f ou ao espaço de busca, tornando a DE notavelmente robusta a transformações de variáveis.

3.1 Estratégia DE/rand/1/bin

Esta é a variante canônica e o ponto de partida recomendado. Para cada indivíduo **alvo** $\boldsymbol{x}_i$ na população, a DE executa três passos:

Passo 1 — Mutação diferencial: seleciona três índices r_1, r_2, r_3 distintos entre si e distintos de i; cria um **vetor mutante**:

$$\boldsymbol{v}_i = \boldsymbol{x}_{r_1} + F\left(\boldsymbol{x}_{r_2} - \boldsymbol{x}_{r_3}\right)$$

onde $F \in (0, 2]$ é o **fator de escala diferencial**.

Passo 2 — Cruzamento binomial: cria o **vetor candidato** $\boldsymbol{u}_i$ combinando $\boldsymbol{v}_i$ e $\boldsymbol{x}_i$:

$$u_{i,d} = \begin{cases} v_{i,d} & \text{se uniforme}(0,1) \leq C_R \text{ ou } d = d_{\text{rand}} \\ x_{i,d} & \text{caso contrário} \end{cases}$$

onde $C_R \in [0,1]$ é a **taxa de cruzamento** e d_{rand} é uma dimensão sorteada que garante que o candidato herde ao menos um componente do mutante.

Passo 3 — Seleção gananciosa:

$$\boldsymbol{x}_i^{t+1} = \begin{cases} \boldsymbol{u}_i & \text{se } f(\boldsymbol{u}_i) \leq f(\boldsymbol{x}_i) \\ \boldsymbol{x}_i & \text{caso contrário} \end{cases}$$

Algoritmo — Evolução Diferencial — DE/rand/1/bin

```
DE(f, N_pop, F, C_R, max_gen):
  P ← {x_1, ..., x_{N_pop}}   // população inicial uniforme em [x_min, x_max]
  avaliar f(x_i) para todo i
  s* ← argmin f(P)

  para g = 1 até max_gen:
    para i = 1 até N_pop:
      r1, r2, r3 ← amostra_sem_reposicao({1..N_pop} \ {i})
      v ← x_{r1} + F * (x_{r2} - x_{r3})
      v ← clipar(v, x_min, x_max)

      d_rand ← uniforme_inteiro(1, n)
      para d = 1 até n:
        se uniforme(0,1) ≤ C_R ou d = d_rand:
          u[d] ← v[d]
        senão:
          u[d] ← x_i[d]

      se f(u) ≤ f(x_i):
        x_i ← u
        se f(u) < f(s*): s* ← u

  retorna s*
```

3.2 Estratégia DE/best/1/bin

Uma variante de alta pressão seletiva substitui $\boldsymbol{x}_{r_1}$ pelo melhor indivíduo da população atual $\boldsymbol{x}_{\text{best}}$:

$$\boldsymbol{v}_i = \boldsymbol{x}_{\text{best}} + F\left(\boldsymbol{x}_{r_2} - \boldsymbol{x}_{r_3}\right)$$

Isso acelera a convergência ao puxar perturbações em direção à melhor solução conhecida, mas reduz a exploração e aumenta o risco de convergência prematura em problemas multimodais.

Aspecto	DE/rand/1/bin	DE/best/1/bin
Exploração	Alta	Baixa
Convergência	Mais lenta	Mais rápida
Risco de ótimo local	Menor	Maior
Indicada para	Paisagens rugosas e multimodais	Unimodais ou orçamento muito restrito

Table 2: Comparação entre variantes da Evolução Diferencial.

3.3 Calibração de F e C_R

Fator de escala F: controla o tamanho do passo de mutação. Valores baixos ($F < 0.4$) produzem perturbações pequenas — explotação intensa mas risco de estagnação. Valores altos ($F > 1.0$) produzem saltos grandes — boa exploração mas dificuldade de refinamento. A faixa $F \in [0.5, 0.9]$ funciona bem na maioria dos problemas; $F = 0.8$ é o ponto de partida canônico de Storn & Price.

Taxa de cruzamento C_R: controla quantas dimensões são herdadas do mutante. C_R alto (próximo de 1) $\longrightarrow$ candidato $\approx$ mutante (forte perturbação); C_R baixo (próximo de 0) $\longrightarrow$ candidato $\approx$ alvo com poucas dimensões trocadas. Para variáveis **altamente correlacionadas**, C_R baixo (≈ 0.1) preserva melhor a estrutura. Para variáveis independentes, C_R alto (≈ 0.9) acelera a convergência.

> **Observação**
> **Estratégia JADE (Zhang & Sanderson, 2009).** Uma variante adaptativa que atualiza F e C_R durante a execução com base no histórico de movimentos bem-sucedidos. Elimina a necessidade de calibração manual e é competitiva com versões estáticas bem calibradas. Recomendada quando não há conhecimento prévio do problema.

3.4 Tratamento de restrições na DE

A DE opera nativamente em espaços contínuos sem restrições. Três estratégias são comuns:

Reparo de solução: projetar $\boldsymbol{u}$ de volta ao espaço viável após a mutação. Para restrições de caixa ($x_d \in [x_d^{\min}, x_d^{\max}]$), basta clipar: $u_d = \max(x_d^{\min}, \min(x_d^{\max}, u_d))$. Para restrições gerais, o reparo é mais complexo e problema-específico.

Penalização: adicionar à função objetivo um termo que penaliza violações. Simples de implementar mas requer calibração cuidadosa do coeficiente de penalidade — o mesmo desafio visto nas Aulas 3 e 4.

Torneio com viabilidade (Deb, 2000): na seleção gananciosa, uma solução viável sempre derrota uma inviável; entre duas inviáveis, vence a com menor violação total; entre duas viáveis, vence a com menor f. Não requer calibração de penalidade e generaliza bem para diferentes classes de restrições.

4 Estudo de Caso: Benchmarks CEC em Dimensão Moderada

Para avaliar AG e DE em condições controladas e reprodutíveis, usamos as funções *benchmark* do CEC (*Congress on Evolutionary Computation*) — conjunto padronizado que cobre as principais estruturas de paisagem encontradas em problemas reais: unimodal suave, multimodal com bacias regulares, multimodal rugosa, e separável vs. não-separável.

Fio Condutor — Knapsack e TSP
Os benchmarks contínuos complementam os problemas combinatórios do fio condutor: o Knapsack (binário, Aulas 1–4) e o TSP (permutacional, Aulas 3–4) exercitam representação discreta e operadores combinatórios. As funções CEC exercitam representação real, perturbação diferencial e o comportamento de AG e DE em paisagens contínuas. Juntos, os três problemas cobrem as principais classes de estrutura combinatória e contínua que o aluno encontrará em aplicações reais.

4.1 As quatro funções de referência

Todas definidas em $n = 10$ dimensões, $\boldsymbol{x} \in [-5, 5]^n$, minimização:

$\boldsymbol{f_1}$ **— Esfera (unimodal, separável):** $f_1(\boldsymbol{x}) = \sum_{i=1}^{n} x_i^2$ Ótimo global em $\boldsymbol{x}^* = \mathbf{0}$, $f_1^* = 0$. Referencial de problema fácil — qualquer algoritmo razoável deve convergir.

$\boldsymbol{f_2}$ **— Rosenbrock (unimodal, não-separável):** $f_2(\boldsymbol{x}) = \sum_{i=1}^{n-1} \left[100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2\right]$ Ótimo global em $\boldsymbol{x}^* = \mathbf{1}$, $f_2^* = 0$. O vale curvo e estreito ("banana") é difícil para algoritmos que ignoram correlação entre variáveis — um diagnóstico direto da qualidade dos operadores.

$\boldsymbol{f_3}$ **— Rastrigin (multimodal, separável):** $f_3(\boldsymbol{x}) = 10n + \sum_{i=1}^{n} [x_i^2 - 10\cos(2\pi x_i)]$ Ótimo global em $\boldsymbol{x}^* = \mathbf{0}$, $f_3^* = 0$. Aproximadamente 10^n mínimos locais regularmente espaçados. O problema-padrão para avaliar capacidade de escapar de ótimos locais.

$\boldsymbol{f_4}$ **— Ackley (multimodal, quase-separável):** $f_4(\boldsymbol{x}) = -20\exp\left(-0.2\sqrt{\frac{1}{n}\sum x_i^2}\right) - \exp\left(\frac{1}{n}\sum \cos(2\pi x_i)\right) + 20 + e$ Ótimo global em $\boldsymbol{x}^* = \mathbf{0}$, $f_4^* = 0$. Combina uma bacia larga e suave com oscilações de alta frequência sobrepostas — paisagem que engana algoritmos com passo de busca fixo.

4.2 Protocolo experimental

Todos os métodos usam o mesmo orçamento: NFE $= 50,000$ avaliações, 30 execuções independentes com sementes registradas. O AG usa representação real com BLX-$\alpha = 0.5$ e mutação polinomial ($\eta_m = 20$); a DE usa DE/rand/1/bin.

4.3 Resultados

Exemplo
AG vs. DE em quatro funções benchmark, $n = 10$ dimensões.

Função	Método	$\overline{f^*}$	σ_{f^*}	Melhor	Sucesso
f_1 Esfera	AG $(N = 50,\ G = 1000)$	1.2×10^{-8}	3.1×10^{-9}	4.0×10^{-9}	100%
	DE $(F = 0.8,\ C_R = 0.9)$	8.3×10^{-15}	2.1×10^{-15}	2.9×10^{-15}	100%
f_2 Rosenbrock	AG $(N = 50,\ G = 1000)$	4.82	2.14	1.23	37%
	DE $(F = 0.8,\ C_R = 0.9)$	0.31	0.18	0.04	93%
f_3 Rastrigin	AG $(N = 50,\ G = 1000)$	18.7	6.2	7.9	23%
	DE $(F = 0.8,\ C_R = 0.9)$	4.1	2.8	0.0	67%
f_4 Ackley	AG $(N = 50,\ G = 1000)$	1.83	0.74	0.21	40%
	DE $(F = 0.8,\ C_R = 0.9)$	0.12	0.09	0.00	87%

Table 3: Comparação AG vs. DE nas quatro funções benchmark (30 execuções, NFE = 50 000). "Sucesso" = fração de execuções com $f^* \leq 0.1$ (exceto Esfera: $f^* \leq 10^{\{-6\}}$).

Em f_1 ambos convergem, mas a DE atinge precisão várias ordens de grandeza maior — reflexo do mecanismo diferencial que escala automaticamente o passo de busca. Em f_2 (Rosenbrock), a DE supera dramaticamente o AG: o vale curvo exige perturbações correlacionadas entre dimensões, que o vetor diferencial $\boldsymbol{x}_{r_2} - \boldsymbol{x}_{r_3}$ captura naturalmente quando a população já reside no vale. O BLX-α do AG, sendo separável por dimensão, não explora essa correlação.

Em f_3 (Rastrigin) e f_4 (Ackley), a multimodalidade afeta ambos, mas a DE mantém vantagem clara. **Teste de Friedman** sobre os 4 problemas × 2 métodos: $p < 10^{\{-5\}}$; post-hoc de Nemenyi confirma DE $\succ$ AG em f_2, f_3 e f_4 ao nível de 5%. Em f_1, a diferença é estatisticamente significativa mas operacionalmente irrelevante — ambos encontram o ótimo.

Diagnóstico de convergência: a variância da população do AG colapsa tipicamente na geração 200–300 (de 1000) em f_3 — convergência prematura antes de explorar suficientemente o espaço multimodal. A DE mantém diversidade até a geração 600–700 pelo mecanismo diferencial auto-regulado.

5 Síntese e Próximos Passos

Esta aula introduziu os dois primeiros algoritmos populacionais do módulo. O AG traz a inspiração biológica que popularizou os algoritmos evolutivos: representação flexível, recombinação de blocos construtivos e controle de diversidade por elitismo e niching. A DE oferece um mecanismo de perturbação diferencial auto-adaptativo que é notavelmente eficiente em otimização contínua — a superioridade nos benchmarks CEC não é acidental, mas reflete a adequação do vetor diferencial a paisagens onde correlações entre variáveis importam.

A comparação nos benchmarks revelou que a DE/rand supera o AG com BLX-α no mesmo orçamento de avaliações em 3 das 4 funções testadas. Isso não é uma lei geral — o NFL proíbe tal afirmação — mas indica que, para paisagens contínuas com dependências entre variáveis, o mecanismo diferencial é mais informativo do que o cruzamento independente por dimensão.

Na **Aula 6**, introduziremos PSO e ACO — dois algoritmos inspirados em comportamento coletivo. O PSO é altamente eficiente em problemas contínuos; o ACO é o método de referência para problemas combinatórios como o TSP, onde veremos uma perspectiva radicalmente diferente de como uma população pode aprender a estrutura do problema.

6 Exercícios

Exercício

1 (Representação e operadores). Para o **Problema da Mochila com $n = 20$ itens** (instância da Aula 1), implemente um AG com representação binária, seleção por torneio ($k = 2$), cruzamento de um ponto ($p_c = 0.8$), mutação bit-flip ($p_m = \frac{1}{n}$) e elitismo ($e = 1$). Execute 30 rodadas com NFE $= 10,000$. Compare com os resultados da busca aleatória e da heurística gulosa das Aulas 1 e 2. O AG supera ambos estatisticamente (Wilcoxon)? A que atribui a diferença de desempenho?

Exercício

2 (Convergência prematura). Implemente o AG do Exercício 1 aplicado à função Rastrigin 10D ($\boldsymbol{x} \in [-5, 5]^{\{10\}}$, NFE $= 50,000$, $N_p = 50$, $N_g = 1000$). (a) Plote a evolução da variância da população ao longo das gerações — em que geração tipicamente colapsa? (b) Adicione reinjeção aleatória: os 10 piores indivíduos são substituídos por soluções aleatórias a cada 50 gerações. (c) Compare custo médio e taxa de sucesso ($f^* \leq 0.1$) com teste de Wilcoxon. A reinjeção resolve o problema de convergência prematura ou apenas o adia?

Exercício

3 (Calibração da DE). Para a função Rastrigin 10D, realize um estudo de sensibilidade da DE/rand/1/bin variando $F \in \{0.3, 0.5, 0.8, 1.0\}$ e $C_R \in \{0.1, 0.5, 0.9\}$ (grade $4 \times 3 = 12$ configurações, NFE $= 30,000$, 30 execuções cada). Produza um mapa de calor do $\overline{f^*}$ em função de (F, C_R) e identifique a região ótima de parâmetros. O resultado está de acordo com as recomendações de Storn & Price (1997)?

Exercício

4 (AG vs. DE — Comparação sistemática). Reproduza o experimento da Table 3 para as quatro funções f_1–f_4 com $n = 10$, NFE $= 50,000$ e 30 execuções independentes. (a) Reporte a tabela completa de métricas ($\overline{f^*}$, σ, melhor, taxa de sucesso). (b) Aplique o teste de Friedman sobre os resultados de todos os problemas e métodos; use Nemenyi como post-hoc. (c) Plote as curvas de convergência médias com IC 95% para f_3 (Rastrigin) — onde as curvas se separam? O que isso revela sobre a diferença de comportamento dos dois algoritmos?

Leituras recomendadas:
• Holland, J. H. **Adaptation in Natural and Artificial Systems**. University of Michigan Press, 1975. *(obra fundadora dos AGs)*
• Storn, R., Price, K. Differential evolution — A simple and efficient heuristic for global optimization over continuous spaces.

Journal of Global Optimization, 11(4), 341–359, 1997. *(artigo original da DE — leitura obrigatória)*
• Deb, K., Agrawal, R. B. Simulated binary crossover for continuous search space. **Complex Systems**, 9(2), 115–148, 1995.
• Zhang, J., Sanderson, A. C. JADE: Adaptive differential evolution with optional external archive. **IEEE Transactions on Evolutionary Computation**, 13(5), 945–958, 2009.
• Liang, J. J. et al. Problem definitions and evaluation criteria for the CEC 2005 special session on real-parameter optimization. Technical Report, Nanyang Technological University, 2005. *(definição das funções benchmark)*